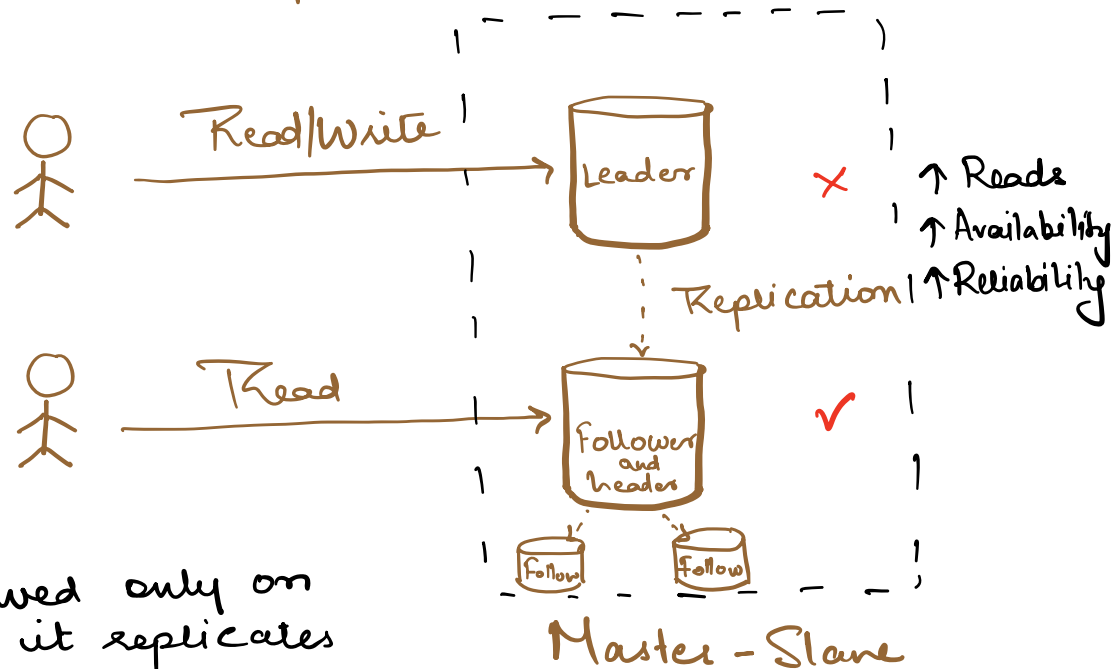
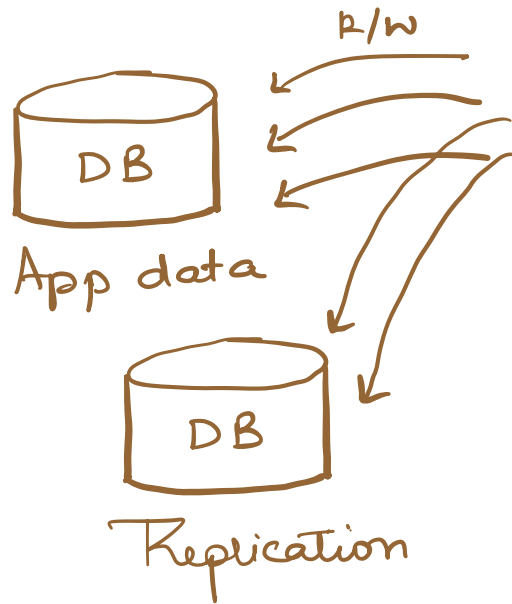


# Replication & Sharding

## REPLICATION



✗ Writes allowed only on leader as it replicates that data to Followers whereas writes does not happen to follower as it does not replicate the data to the leader. Thus, this allows to Scale Reads operation

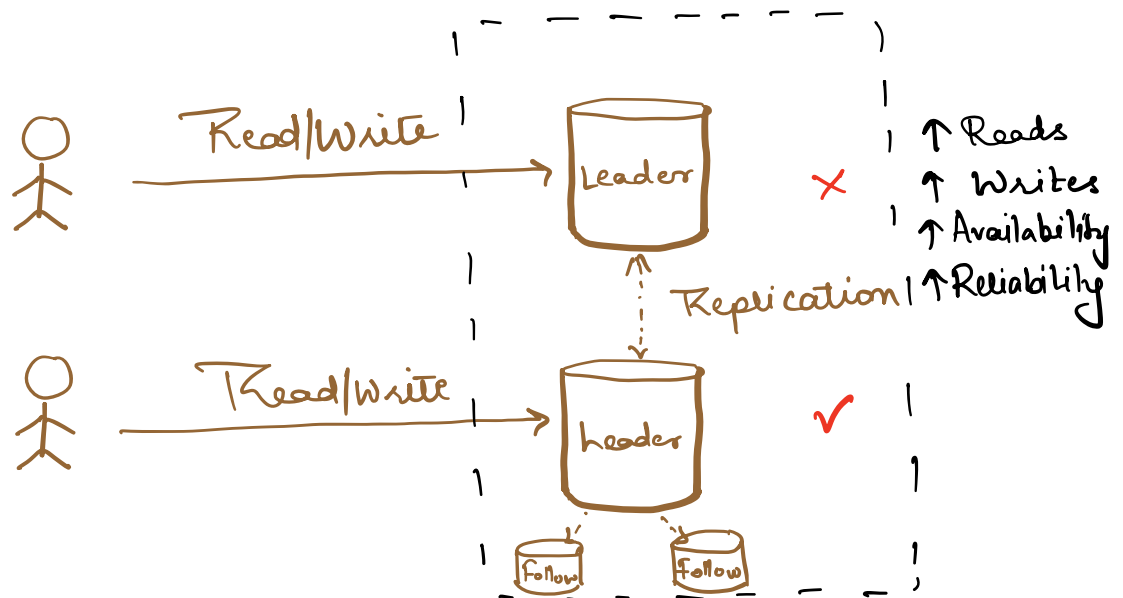
## \* Sync vs Async Replication

Async :- When we don't have to replicate immediately like the client writes data on leader and then after some point it will be replicated to follower probably through a schedule

Issue :- Other client reading data from follower can not see updated data for a short amount of time meaning Inconsistent/Stale data

Sync :- Everytime a write transaction happens on a leader that is immediately replicated to the follower. No inconsistent data/stale data available for a reader

Issue :- Latency, the write transaction will not complete until replicated to the follower.

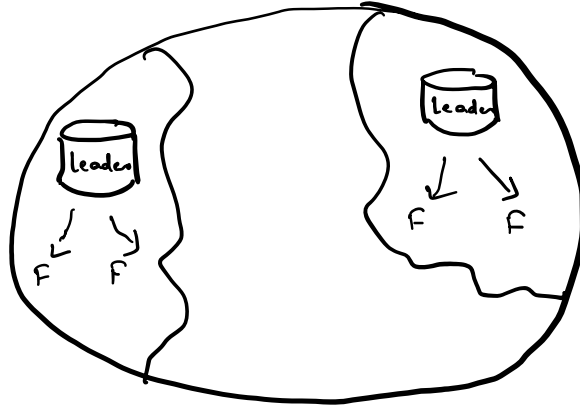


### Master - Master/Multi-Master

X: Though ↑ Reads & Writes can be obtained but it can go out of sync as multiple writers across different DB happen and then they need to swap

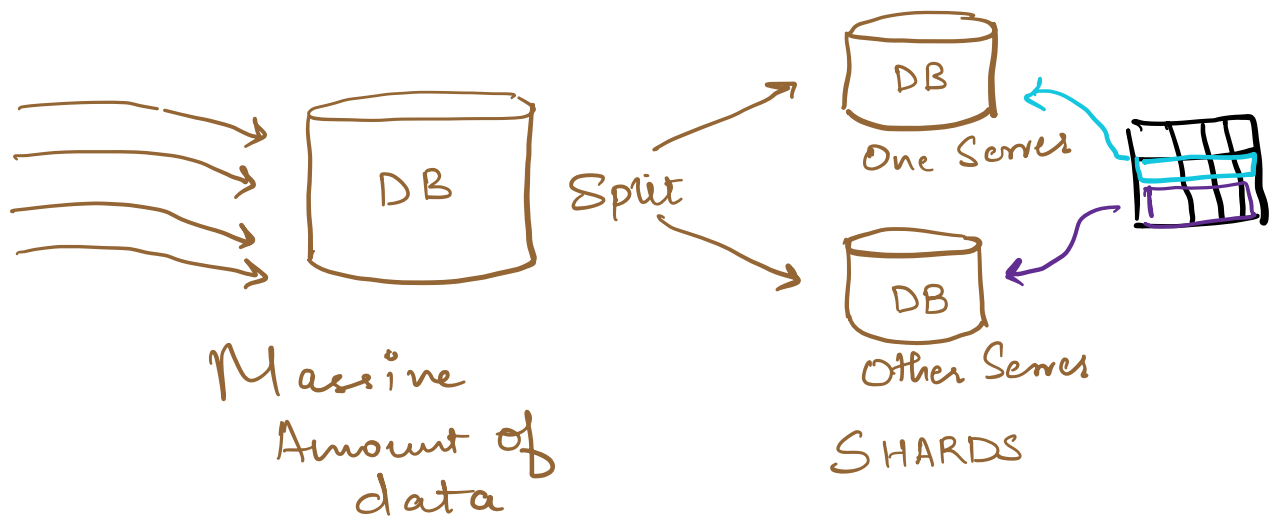
Creates ↓ Consistency if we do Async replication  
Also, if we do sync then it will surely  
much ↑ Latency

Example:- Multi leader using Async



So one part of the world can have  
same data as followed by their leader  
the other part of the world can have  
data as per their leader meaning  
Eventual Consistency can be allowed

## SHARDING

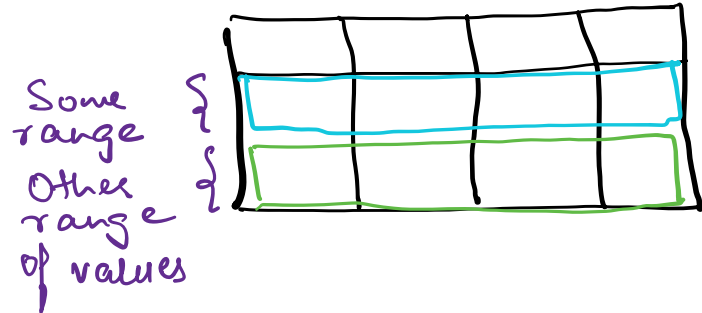


## Benefits

- More traffic can be handled
- Query Results will be faster through SHARDS

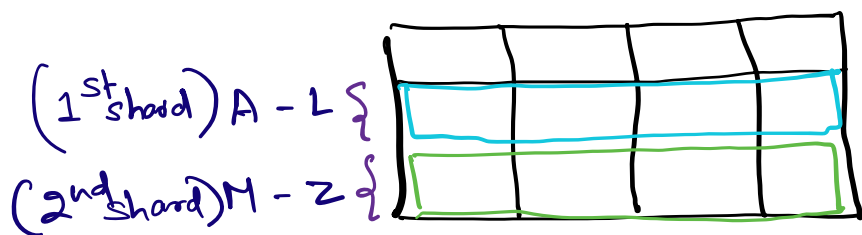
How do we decide what half of the data goes where?

## 1) Shard Key - Range Based Sharding



- For Range based we use some range of values to decide the data will go to which shard. Shard Key/PK is used here to split the data based on range values

## 2) Shard Key - Last Name Sharding



- Say if your last name starts from character between A-L your data will go to 1<sup>st</sup> shard and if it falls between M-Z then 2<sup>nd</sup> shard

~~X~~ Running Joins on 2 halves of table is very difficult. Consistency can be an issue if we have data in 1 table

corresponds to the data in 2<sup>nd</sup> table  
how do we map that data if tables are  
in different shards

### 3) Shard Key-Hash based Sharding

- This is a use case for Consistent Hashing (CH can be used to shard data)

Note:- SQL DB like MySQL, Postgres do not have sharding by default (need to build custom logic)

NoSQL DB will have sharding by default since they can be horizontally scaled and does not care of consistency rather look for eventually consistency also constraints are difficult to achieve.

Basically No ACID property followed in NoSQL so sharding is default.